



Smart HAS

Monitoramento de Hipertensão Arterial Sistêmica

App mobile (React Native) • API (Java/Spring Boot) • Painel web (Angular)

Integrantes: Gabriel Garuti Paiva Cracco — RM 554866

O problema

Hipertensão é silenciosa e exige acompanhamento contínuo

Contexto

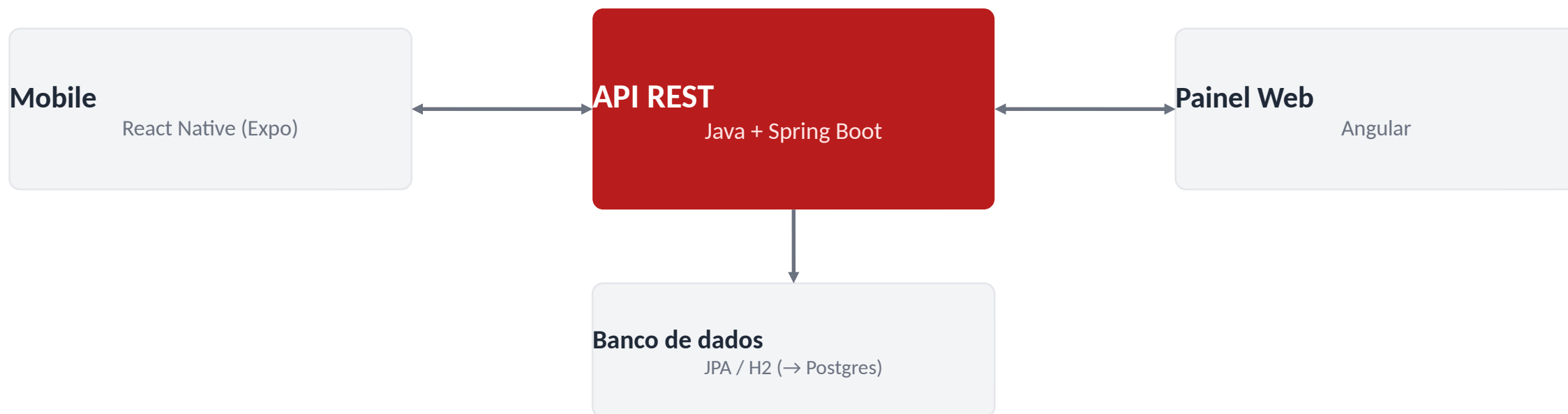
- A hipertensão (HAS) costuma ser assintomática e subdiagnosticada.
- O acompanhamento depende de medições frequentes e organizadas.
- Pacientes carecem de retorno claro sobre o que cada valor significa.
- Faltam alertas e orientação sobre quando procurar ajuda.

A proposta Smart HAS

- Registro rápido de medições (sistólica/diastólica).
- Classificação automática: Normal, Elevada ou Hipertensão.
- Nível de risco e recomendações personalizadas.
- Indicação da unidade de saúde mais próxima.

Arquitetura

Uma API central e dois clientes



Autenticação JWT • JSON • Swagger • Thymeleaf

Parte 1 — Mobile em React Native

Migração do app Flutter para React Native (Expo)

Por que migrar?

- Exercita os componentes-alvo: View, Text, Image, Button.
- Navegação com React Navigation (stack + tabs).
- Consome a MESMA API do painel web (arquitetura unificada).
- Mantém o contrato de dados do app original (migração fiel).

Telas implementadas

- Login com token persistido (AsyncStorage + Context).
- Início: última medição, classificação e nível de risco.
- Histórico de medições (FlatList).
- Nova medição (formulário) e Unidades de saúde.

Parte 2 — API em Java + Spring Boot

REST em camadas, segura e documentada

Recursos

- Spring MVC (REST/JSON) + Thymeleaf (página server-side).
- Camadas: Controller, Service, Repository, Domain.
- Autenticação JWT e autorização por perfil (USER/ADMIN).
- JPA/Hibernate com banco H2 (troca simples p/ Postgres).
- Validação (Bean Validation) e erros centralizados.
- Documentação Swagger/OpenAPI.

Principais endpoints

- POST /auth/register • POST /auth/login
- GET/POST/PUT/DELETE /measurements
- GET /units • POST/PUT/DELETE /units (ADMIN)
- GET /recommendations (apoio à decisão)
- GET /health • /swagger-ui.html

Parte 3 — Painel web em Angular

Consome a mesma API via HttpClient

Conceitos aplicados

- Data binding: {{ }}, [], () e [()] com ngModel.
- Diretivas: *ngIf e *ngFor.
- Formulários com FormsModule / ngModel.
- Rotas e navegação (routerLink).
- Serviços injetáveis + interceptor de JWT.

Telas

- /login — autenticação.
- /dashboard — risco e recomendações.
- /measurements — CRUD de medições.
- /units — gestão de unidades (ADMIN).

Apoio à decisão & Demonstração

AI Logistics Extension

Como funciona

- Analisa o histórico e conta as classificações.
- Calcula o nível de risco: BAIXO, MODERADO ou ALTO.
- Gera recomendações textuais personalizadas.
- Sugere a unidade ativa mais próxima (Haversine).

Credenciais de demonstração

- admin@smarthas.com / admin123 (perfil ADMIN)
- paciente@smarthas.com / 123456 (perfil USER)
- Backend: <http://localhost:8080>
- Swagger: <http://localhost:8080/swagger-ui.html>
- Painei: <http://localhost:4200>

Roadmap tecnológico

Evolução do Smart HAS

Fase anterior

- App Flutter (web).
- Backend Python/FastAPI + SQLite.
- Mapa (Google Maps) e notificações.

Esta fase

- Mobile migrado p/ React Native.
- API reescrita em Java/Spring Boot.
- Novo painel em Angular.
- Uma API, dois clientes.

Próximas fases

- Deploy em nuvem + banco gerenciado.
- Notificações push reais.
- Build nativo (EAS Build).
- Apoio à decisão avançado.

Entrega & Links

Repositório GitHub: (inserir link)

Vídeo de demonstração (YouTube, não listado): (inserir link)

Documentação completa em PDF incluída na entrega.



Obrigado!

Smart HAS — cuidando da sua pressão com tecnologia